

Числа фибоначчи. Рекурсия

Вопросы рекурсии: Рекуррентные формулы и реализация

Последовательность чисел Фибоначчи F_n задаётся линейным рекуррентным соотношением:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ где } n \geq 2, \quad n \in \mathbb{Z}$$

На Западе эта последовательность была исследована Леонардо Пизанским, известным как Фибоначчи, в его труде «Книга абака» (1202). Он рассматривает развитие идеализированной (биологически нереальной) популяции кроликов, где условия таковы: изначально дана новорождённая пара кроликов (самец и самка); со второго месяца после своего рождения кролики начинают спариваться и производить новую пару кроликов, причём уже каждый месяц; кролики никогда не умирают, — а в качестве искомого выдвигает количество пар кроликов через год:

1. начале первого месяца есть только одна новорождённая пара (1);
2. конце первого месяца по-прежнему только одна пара кроликов, но уже спарившаяся (1);
3. конце второго месяца первая пара рождает новую пару и опять спаривается (2);
4. конце третьего месяца первая пара рождает ещё одну новую пару и спаривается, вторая пара только спаривается (3).
5. конце четвёртого месяца первая пара рождает ещё одну новую пару и спаривается, вторая пара рождает новую пару и спаривается, третья пара только спаривается (5).

В конце n -го месяца количество пар кроликов будет равно количеству пар в предыдущем месяце плюс количеству новорождённых пар, которых будет столько же, сколько пар было два месяца назад, то есть $F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}$. Возможно, эта задача также оказалась первой, моделирующей экспоненциальный рост популяции.

Дальше можно смотреть по ссылке:
[Числа Фибоначчи](#)

Образец кода на лисп (sbcl):

```
;;; Programm for calculation Fibonacci sequences
;;; https://www.jdoodle.com/execute-clisp-online

;;; FIB computes the the Fibonacci function in the traditional
;;; recursive way.

;; Fibonacci calc function.
(defun fib(n)
  (check-type n integer)
  ;; At this point we're sure we have an integer argument.
  ;; Now we can get down to some serious computation.
  (cond ((= n 0) 0)
        ((= n 1) 1)
        (t ( + (fib (- n 2))      ; The traditional formula
                 (fib (- n 1)) ))) ; is fib[n-1]+fib[n-2].
  )

(defun cli/parameter()
  ;;Get command line argument"
  (let (
    (args sb-ext:*posix-argv*))
    (car (cdr args)))
  )

(defun makeDomain(num acc)
  ;;Make domain's interval from 0 to num "
  (cond ((eq num 0) acc)
        (t (makeDomain (- num 1) (cons num acc))))
  )

;; call fib function for interval
;; input ./exec4.lisp <n = 25>
;; main print
(defun main()
  (terpri)
  (format T "~a" (mapcar 'fib (makeDomain (parse-integer (cli/parameter)) nil)))
  )
```

```
(main)
```

```
;;-----
```

Образец кода на haskell:

```
module Math where
```

```
fib :: Int -> Int
```

```
fib 0 = 0
```

```
fib 1 = 1
```

```
fib n = fib (n-1) + fib (n-2)
```

```
-- The memoized version is much faster. Try memoized_fib 10000.
```

```
-- http://wiki.haskell.org/Memoization
```

```
-- http://lin-techdet.blogspot.com/2015/07/haskell.html
```

```
memoized_fib :: Int -> Integer
```

```
memoized_fib = (map fib [0 ..] !!)
```

```
    where fib 0 = 0
```

```
          fib 1 = 1
```

```
          fib n = memoized_fib (n-2) + memoized_fib (n-1)
```

Main.hs:

```
module Main where
```

```
import System.Environment
```

```
import Data.List
```

```
import Math
```

```
{- https://stackoverflow.com/questions/1105765/generating-fibonacci-numbers-in-haskell -}
```

```
getIntArg :: IO Int
```

```
getIntArg = fmap (read . head) getArgs
```

```
main = do
```

```
    args <- getIntArg                -- IO [Int]
```

```
    putStrLn "The arguments are:"
```

```
    print (fib args)
```

Ниже представлен результат для $n = 20$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181,
6765, 10946, 17711